Pimpri Chinchwad Education Trust's

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

DEPARTMENT OF INFORMATION TECHNOLOGY

[Insert college logo here — set logo_path in report_config.json]

MINI PROJECT REPORT

on

Analysis of Air Pollution Patterns and Pollution Hotspots Across India Using Big Data Technologies

Submitted by

Name of Student	PRN
[Student Name 1]	[PRN 1]
[Student Name 2]	[PRN 2]
[Student Name 3]	[PRN 3]
[Student Name 4]	[PRN 4]

Div: [Div]

Academic Year: 2026-27

DRAFT TEMPLATE — result sections are pending a pipeline run on the real dataset.

TABLE OF CONTENTS

1. Introduction 4
1.1 Background 4
1.2 Problem Statement 4
1.3 Objectives 4
1.4 Scope 4
2. Dataset and Preprocessing 4
2.1 Dataset Source and Collection 4
2.2 Important Attributes 5
2.3 Observed Dataset Coverage 5
2.4 Schema-Aware Loading 5
2.5 Cleaning and Standardisation 6
2.6 Cleaning Counts and Provenance 6
2.7 Collection Limitations 6
3. System Architecture and Technologies 7
3.1 Overall Architecture 7
3.2 Layer Responsibilities 7
3.3 Technology Roles 7
3.4 Execution and Storage Boundaries 8
3.5 Result Data Contract 8
4. Batch Analysis Using Apache Spark 8
4.1 Transformations and Actions 8
4.2 Counting Rules and Denominators 8
4.3 Output Preparation 8
4.4 Pollutant Coverage 8
4.5 City Comparison 8
4.6 State Comparison 8
4.7 Monthly Trend 8
4.8 Seasonal Pattern 8
4.9 Hourly Pattern 9
4.10 Pollution Hotspots 9
4.11 Abnormal Pollution Levels 9
4.12 Air Quality Index 9
4.13 Insights and Recommendations 9
5. Dashboard and Visualization 9
5.1 Streamlit Application 9
5.2 Metric Cards 9
5.3 Filters 9
5.4 Dashboard Screenshots 9
5.5 Downloads 10
5.6 Visualization Provenance 10
6. Storage and Integration 10
6.1 Parquet Storage Layout 10
6.2 Why Parquet 10
6.3 Apache Cassandra (Not Implemented) 10
6.4 Reliability Considerations 10
7. Local Deployment and Execution 11
7.1 Development Environment 11
7.2 Prepare the Data 11
7.3 Run the Spark Pipeline 11
7.4 Start the Dashboard 11
7.5 Build the Report and Run Tests 11
8. Testing and Validation 11
8.1 Validation Approach 11

8.2 Timestamp Handling 11
8.3 Test Results 11
8.4 Data-Quality Checks on Real Data 12
8.5 Limitations of Validation 12
9. Conclusion and Future Scope 12
9.1 Project Summary 12
9.2 Future Scope 12
10. References 13

1. Introduction

1.1 Background

Air pollution is one of India's most significant environmental health risks. The Central Pollution Control Board (CPCB) operates a Continuous Ambient Air Quality Monitoring (CAAQM) network that reports pollutant concentrations hourly. Combined over many years and stations, these readings form a dataset large enough to need distributed processing for cleaning, aggregation and anomaly screening.

This Big Data Analytics mini project uses Apache Spark (PySpark) to ingest the India Air Quality Database, clean it, and compute city, state, seasonal, hourly, hotspot, anomaly and AQI summaries stored as Parquet. A Streamlit dashboard serves the saved results.

1.2 Problem Statement

Develop a reproducible, laptop-runnable pipeline that analyses hourly air-quality readings across India, identifies pollution hotspots and abnormal pollution levels, compares cities and states, characterises monthly, seasonal and diurnal patterns, and presents the results in an interactive dashboard.

1.3 Objectives

- Ingest and validate real CPCB-derived hourly data with Apache Spark using an explicit schema contract.
- Clean, standardise units, handle invalid values and deduplicate readings; report data-quality counts.
- Analyse PM2.5, PM10, NO2, SO2, CO, ozone and the other available pollutants.
- Rank cities/states and flag hotspot stations and abnormal pollution days using documented statistical rules.
- Compute the CPCB National AQI only where its validity rules are met.
- Provide an interactive Streamlit/Plotly dashboard with CSV downloads, plus logging and automated tests.

1.4 Scope

The scope is the date range downloaded and analysed locally; coverage figures are filled in after the pipeline run. Apache Cassandra was not implemented (Section 6).

2. Dataset and Preprocessing

2.1 Dataset Source and Collection

The data come from the India Air Quality Database published by the XKDR Forum (https://airquality.xkdr.org/). According to the project's site (checked 20 October 2026), it provides hourly readings from every station in CPCB's CAAQM network plus five US Embassy monitors: about 196.5 million readings from 558 stations and 15 pollutants, covering January 2009 to March 2026. Data are distributed through a query API and as one sorted Parquet file per month (under half a gigabyte in total).

Licence: Creative Commons Attribution 4.0 (CC BY 4.0), with credit to the India Air Quality Database (XKDR Forum) and the original sources (CPCB CAAQM; US Department of State via AirNow). The data are provided without warranty and are published as received from the networks, which label readings preliminary and not fully validated.

Download strategy for a student laptop: scripts/download_data.py fetches only the requested year(s)/months. A public demo key published on the site can read the 2024 files; a free personal key

(https://airquality.xkdr.org/signup) has no row caps and reads all years. One year of Parquet is a small download relative to the whole archive.

2.2 Important Attributes

Table 1. Source schema (per the project's API documentation) and use in this project.

Attribute	Type	Analytical purpose
station_id	text	Station key (CPCB site number or embassy id); identifies series and station counts.
station_name, state_name, city_name	text	Geography for city/state comparison and hotspot labels.
parameter_name	text	Pollutant (PM2.5, PM10, NO, NO2, NOx, NH3, SO2, CO, Ozone, Benzene, Toluene, Xylene, O_Xylene, MP-Xylene, Eth-Benzene).
unit	text	Harmonised units (µg/m³, mg/m³, ppb); standardised in cleaning.
collected_at	timestamp	Naive timestamp in Indian Standard Time; CPCB readings sit on the hour.
value	double	Measured concentration, as received (no gap filling, outlier removal or calibration).
source	text	cpcb_caaqm or us_embassy.
year, month	int	Hive partition columns of the bulk files; used for partition pruning.
latitude, longitude (stations.parquet)	double	Station coordinates for the hotspot map; some decommissioned stations have none.

2.3 Observed Dataset Coverage

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

2.4 Schema-Aware Loading

read_raw() reads the monthly Parquet files with partition discovery, then enforces a schema contract: required columns (station_id, parameter_name, unit, collected_at, value) must exist or a SchemaError is raised; all columns are cast to canonical types (strings trimmed, value as double, collected_at as timestamp); optional columns that are absent become nulls. Parquet carries its own physical schema, so no inference over the data is needed. The Spark session timezone is Asia/Kolkata because the source timestamps are naive IST, so hour-of-day and date derivations are IST.

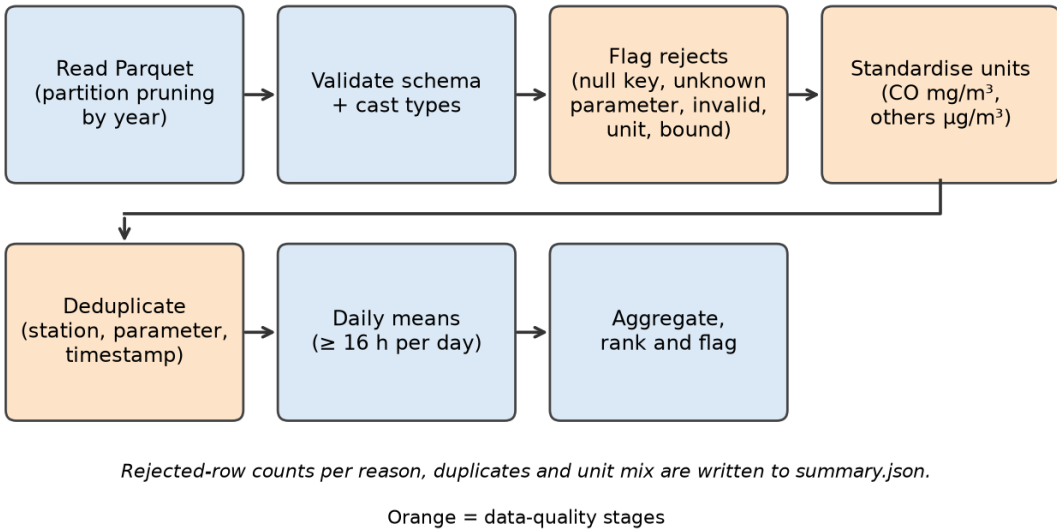


Fig. 1. Spark cleaning and aggregation stages (data-quality stages in orange).

2.5 Cleaning and Standardisation

- Rejected: missing station/parameter/timestamp (null_key_field); parameters outside the known list (unknown_parameter); null, NaN or negative values (invalid_value).
- Units normalised to µg/m³ (CO in mg/m³); µg ↔ mg conversions applied; ppb/ppm rows are quarantined (unit_not_convertible) rather than converted with assumed temperature/pressure.
- Rejected when above a physical plausibility bound per pollutant (above_physical_bound); bounds are configurable assumptions in src/aq/config.py, not standards.
- Deduplicated on (station_id, parameter_name, collected_at); conflicting duplicates are averaged and counted.
- Daily means require ≥ 16 hourly readings (CPCB rule); 8-hour windows for CO and ozone require ≥ 6 readings.
- No gap filling or outlier removal is applied; extreme-but-plausible values are kept and handled by the anomaly module.

2.6 Cleaning Counts and Provenance

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

2.7 Collection Limitations

Monitoring stations are not uniformly distributed and are concentrated in cities; readings are preliminary and not fully validated by the networks; station availability varies over time; and aggregates are not population-weighted. Findings describe monitored locations in the analysed period only.

3. System Architecture and Technologies

3.1 Overall Architecture

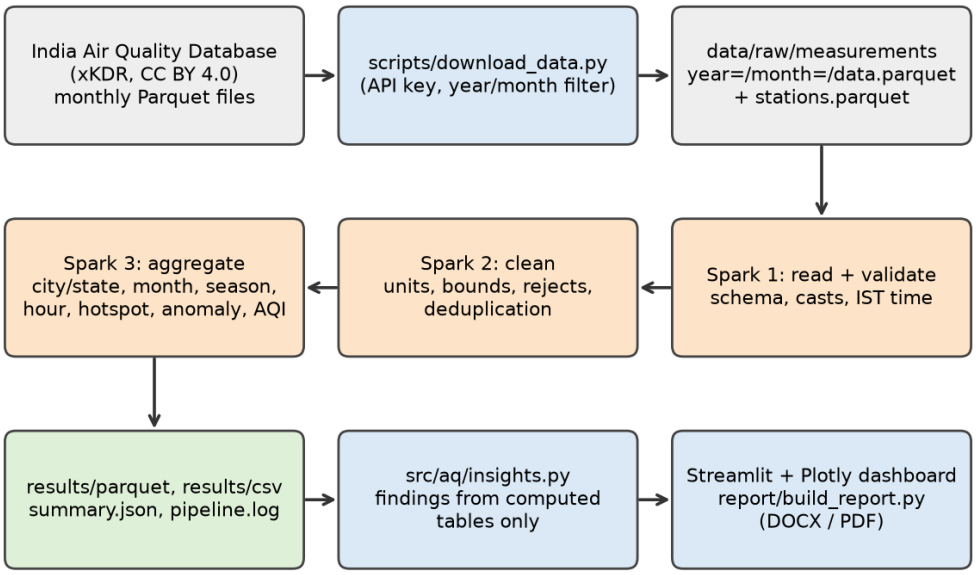


Fig. 2. Architecture: download → Spark validation, cleaning and aggregation → Parquet/CSV → dashboard and report.

3.2 Layer Responsibilities

- Acquisition: scripts/download_data.py (year/month filter, retries, resume by size).
- Processing: src/aq/spark_pipeline.py — all cleaning and aggregation run as Spark DataFrame jobs.
- Storage: Parquet (clean hourly data partitioned by year/month; one Parquet dataset per aggregate) plus CSV exports and summary.json.
- Insights: src/aq/insights.py derives text only from computed tables.
- Presentation: app.py (Streamlit/Plotly) reads the aggregates only; Spark is not run per interaction.

3.3 Technology Roles

Table 2. Technology roles.

Technology	Role
Apache Spark / PySpark 4	Schema enforcement, cleaning, window functions (8-hour averages, ranking), group-by aggregations, percentiles.
Parquet (pyarrow)	Columnar input/output; partition pruning by year.
pandas	Reading small aggregate tables for the dashboard, report and insights.
Streamlit + Plotly	Interactive dashboard, filters, maps, downloads.
Matplotlib + python-docx	Static report charts and the DOCX report.
pytest	Automated unit, integration and dashboard smoke tests.

3.4 Execution and Storage Boundaries

Spark runs in local mode (default local[2], 8 shuffle partitions, 2 GB driver memory; adjustable by command-line flags). Local threads demonstrate Spark's APIs and lazy execution on a laptop; they do not demonstrate cluster scalability, and no throughput benchmark is claimed. Aggregate tables are small and converted to pandas only after Spark has reduced them.

3.5 Result Data Contract

Each aggregate is stored as results/parquet/<name>/ and results/csv/<name>.csv: city_summary, state_summary, monthly_city, monthly_national, seasonal_city, hourly_pattern, hotspots, anomaly_days, anomaly_summary, aqi_station_daily (Parquet only), aqi_city_monthly, aqi_category_city, aqi_dominant_city, data_coverage. summary.json holds run metadata and data-quality counts.

4. Batch Analysis Using Apache Spark

4.1 Transformations and Actions

Transformations (select, filter, withColumn, groupBy, window functions, join) build a lazy plan; actions (write, count, first, collect, toPandas on aggregates) trigger execution. Key operations: groupBy().agg() for daily means with hour counts; percentile_approx for medians and 95th percentiles; Window.rangeBetween(-7h, 0) for 8-hour rolling means of CO and ozone; percent_rank and row_number windows for hotspot ranking; max_by for the dominant AQI pollutant.

4.2 Counting Rules and Denominators

City/state statistics are means of valid station-day means, so every station-day has equal weight; they are not population-weighted and not hourly-reading weighted. Cities and states are ranked only with at least the minimum station-days set in the dashboard (default 30); hotspot ranking needs at least 60 valid days per station. Exceedance percentages use NAAQS 24-hour limits (PM2.5 60, PM10 100, NO2 80, SO2 80, NH3 400 µg/m³) applied to valid daily means only.

4.3 Output Preparation

Every aggregate is written once as Parquet by Spark and exported to CSV for downloads; summary.json and pipeline.log record the run.

4.4 Pollutant Coverage

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.5 City Comparison

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.6 State Comparison

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.7 Monthly Trend

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.8 Seasonal Pattern

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.9 Hourly Pattern

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.10 Pollution Hotspots

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.11 Abnormal Pollution Levels

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.12 Air Quality Index

The database has no AQI column, so the CPCB National AQI is computed: sub-index linear within each band, 24-hour means for PM2.5, PM10, NO2, SO2, NH3 and 8-hour means for CO and ozone; overall AQI = highest sub-index; a station-day needs ≥ 16 hours per pollutant, ≥ 3 pollutants including PM2.5 or PM10. Band tables are implemented from the CPCB NAQI summary; CPCB does not publish the upper bound of the last band, so it is extended by the previous band's width and capped at 500 (assumption).

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

4.13 Insights and Recommendations

[Pending: generated automatically after the pipeline is run on the downloaded dataset — see README.]

5. Dashboard and Visualization

5.1 Streamlit Application

app.py reads the Parquet aggregates and presents eight tabs: Cities & states, Trends, Hourly, Hotspots, Anomalies, AQI, Insights and Data quality. Plotly provides interactive charts; every table has a CSV download button.

5.2 Metric Cards

Cards show stations, cities/states, clean hourly readings, mean daily value of the selected pollutant, most polluted city, percentage of days above NAAQS, hotspot stations, anomalous station-days and valid-AQI station-days.

5.3 Filters

Sidebar filters: pollutant, state, city, month range, season, top-N and minimum station-days. Aggregates are independent marginal tables, so a filter applies only to tables that carry that dimension (for example, the season filter applies to the seasonal view).

5.4 Dashboard Screenshots

[Screenshot pending: capture the dashboard tab and save as docs/screenshots/01_overview.png] — Dashboard overview: metric cards and city/state rankings.

[Screenshot pending: capture the dashboard tab and save as docs/screenshots/02_trends.png] — Dashboard trends: monthly series and seasonal comparison.

[Screenshot pending: capture the dashboard tab and save as docs/screenshots/03_hourly.png] — Dashboard hourly (diurnal) pattern for selected cities.

[Screenshot pending: capture the dashboard tab and save as docs/screenshots/04_hotspots.png] — Dashboard hotspot map and ranked station table.

[Screenshot pending: capture the dashboard tab and save as docs/screenshots/05_anomalies.png] — Dashboard anomaly view.

[Screenshot pending: capture the dashboard tab and save as docs/screenshots/06_aqi.png] — Dashboard AQI categories and dominant pollutants.

[Screenshot pending: capture the dashboard tab and save as docs/screenshots/07_data_quality.png] — Dashboard data-quality tab.

5.5 Downloads

Each table can be downloaded as CSV; the Data-quality tab also offers summary.json. The pipeline additionally writes all aggregates to results/csv/.

5.6 Visualization Provenance

Figures in Section 4 are rendered by report/build_report.py from results/parquet; dashboard screenshots are taken from the running application. Diagrams are drawn by report/diagrams.py.

6. Storage and Integration

6.1 Parquet Storage Layout

Table 3. Storage layout.

Path	Content
data/raw/measurements/year=/month=	Downloaded source Parquet (read-only).
results/parquet/clean_hourly/year=/month=	Cleaned, deduplicated hourly readings in canonical units.
results/parquet/<aggregate>/	One Parquet dataset per aggregate table.
results/csv/<aggregate>.csv	CSV exports for download and the report.
results/summary.json, pipeline.log, insights.md/json	Run metadata, logs, generated insights.

6.2 Why Parquet

Parquet is columnar, compressed and natively read/written by Spark and pandas, which keeps the project runnable on a laptop without a database server.

6.3 Apache Cassandra (Not Implemented)

Cassandra was optional and is not part of this implementation. The workload is batch analytics over a static historical dataset, for which Parquet is sufficient. Cassandra would be appropriate for serving low-latency lookups (for example, latest readings per station) in a future streaming extension; this is future scope only.

6.4 Reliability Considerations

Aggregate tables are overwritten on each run (not versioned), so an interrupted run can leave a mix of old and new tables; re-run the pipeline to restore consistency. The pipeline exits with a non-zero code and a logged message for missing input, schema errors or no valid rows.

7. Local Deployment and Execution

7.1 Development Environment

Python 3.10+, Java 17 or later (required by Spark), 8 GB RAM recommended. Dependencies are in requirements.txt (PySpark 4.0.1, pandas, pyarrow, Streamlit, Plotly, python-docx, matplotlib, pytest).

7.2 Prepare the Data

```
python -m venv .venv && source .venv/bin/activate        # Windows: .venv\Scripts\
activate
pip install -r requirements.txt
python scripts/download_data.py --years 2024
```

7.3 Run the Spark Pipeline

```
python -m src.aq.spark_pipeline --years 2024 --master local[4] --driver-memory 4g
python -m src.aq.insights results
```

7.4 Start the Dashboard

```
streamlit run app.py
```

7.5 Build the Report and Run Tests

```
pytest --junitxml=docs/pytest_report.xml
python report/build_report.py --results results
```

8. Testing and Validation

8.1 Validation Approach

Automated pytest tests verify cleaning rules, unit conversion, deduplication, schema errors, AQI maths (including Spark/Python parity), end-to-end aggregates against hand-computed expectations, hotspot and anomaly detection, insights generation and a Streamlit smoke test (AppTest). The end-to-end tests run on a small deterministic synthetic fixture created in tests/conftest.py, chosen so every expected value can be computed by hand; it is used for testing only and never for reported findings.

8.2 Timestamp Handling

During development, a test exposed a timezone defect: collecting a Spark timestamp into Python shifted it by the machine's UTC offset (5 h 30 min relative to IST). Timestamp bounds in summary.json are therefore formatted inside Spark in the IST session timezone, and the regression test asserts the expected bound.

8.3 Test Results

The suite was executed on 2026-10-20T05:06:25 (development sandbox, Linux): 29 tests, 0 failed, in 39 s.

Table 4. Automated test results (from docs/pytest_report.xml).

Module	Test	Result
test_aqi	test_sub_index_known_values	passed
test_aqi	test_sub_index_invalid_and_cap	passed
test_aqi	test_bands_monotonic	passed

test_aqi	test_categories	passed
test_aqi	test_overall_rules	passed
test_aqi	test_spark_matches_python	passed
test_cleaning	test_unit_code	passed
test_cleaning	test_reject_reasons_and_dedup	passed
test_cleaning	test_co_converted_to_mg	passed
test_cleaning	test_year_filter	passed
test_cleaning	test_schema_error	passed
test_cleaning	test_missing_path	passed
test_cleaning	test_cli_returns_error_code_on_bad_input	passed
test_downloader_and_app	test_download_plan_filters_and_maps	passed
test_downloader_and_app	test_dashboard_runs_without_exception	passed
test_downloader_and_app	test_dashboard_shows_helpful_error_without_results	passed
test_pipeline	test_summary_counts	passed
test_pipeline	test_city_means	passed
test_pipeline	test_state_summary_and_coverage	passed
test_pipeline	test_seasons_and_months	passed
test_pipeline	test_hourly_pattern	passed
test_pipeline	test_hotspots	passed
test_pipeline	test_anomaly_detected	passed
test_pipeline	test_aqi_rules_and_values	passed
test_pipeline	test_outputs_written	passed
test_pipeline	test_insights_from_results	passed
test_report	test_full_report_uses_computed_values	passed
test_report	test_template_marks_results_pending	passed
test_report	test_missing_results_returns_error	passed

These results come from the synthetic test fixture. They validate code logic, not the real dataset; re-run pytest on your machine.

8.4 Data-Quality Checks on Real Data

Every pipeline run records rejected-row counts, duplicates and unit mix (Section 2.6). Real-data values are pending a pipeline run.

8.5 Limitations of Validation

- Download from airquality.xkdr.org was not possible in the development sandbox (network policy); the downloader was written from the published API documentation and unit-tested only for file selection.
- Real-data runtime, memory use and scalability were not measured there; local[2] results on a small fixture do not predict full-year performance.
- AQI band tables and NAAQS values were taken from public summaries of CPCB documents and should be confirmed against the official notifications for formal use.

9. Conclusion and Future Scope

9.1 Project Summary

The project delivers a PySpark pipeline, Parquet storage, a Streamlit dashboard, tests and a report generator. Findings are produced after running the pipeline on the real dataset.

9.2 Future Scope

- Run on multi-year data and add a cluster/Docker-based Spark profile with measured scalability.
- Add Apache Cassandra or another serving store for near-real-time station lookups.
- Population- or area-weighted aggregation and meteorological covariates.
- Forecasting models and source apportionment for hotspot stations.

- Versioned result snapshots and automated monthly refresh.

10. References

1. XKDR Forum, India Air Quality Database, https://airquality.xkdr.org (CC BY 4.0); source: CPCB CAAQM network and US Department of State via AirNow.

2. xKDR/Air-Quality-Database repository and docs/PUBLIC_API.md, https://github.com/xKDR/Air-Quality-Database.

3. Central Pollution Control Board, National Air Quality Index (summary: https://ies.gov.in/arthapedia/concept/national-air-quality-index).

4. National Ambient Air Quality Standards 2009 (summary: https://ies.gov.in/arthapedia/concept/ambient-air-quality-standards-india).

5. Iglewicz, B. and Hoaglin, D., How to Detect and Handle Outliers, ASQC Quality Press, 1993 (modified z-score).

6. Apache Spark / PySpark documentation; Streamlit and Plotly documentation.